

搜索

- DFS
 - 递归
 - 非递归
- BFS
 - 队列
- 特点：
 - 有固定的通用模块，只要根据具体情况修改状态的存储方式即可

8皇后代码运行分析

Procedure DFS(i:integer); //放第i个皇后

{搜索第i行皇后的位置}

Var

j:integer;

begin

if i=n+1 then 输出方案并返回;

for j:=1 to 8 do

begin

if 皇后能放在第i行第j列的位置 then

begin

放置第i个皇后;

对放置皇后的位置进行标记;

DFS (i+1)

对放置皇后的位置释放标记;

end;

end;

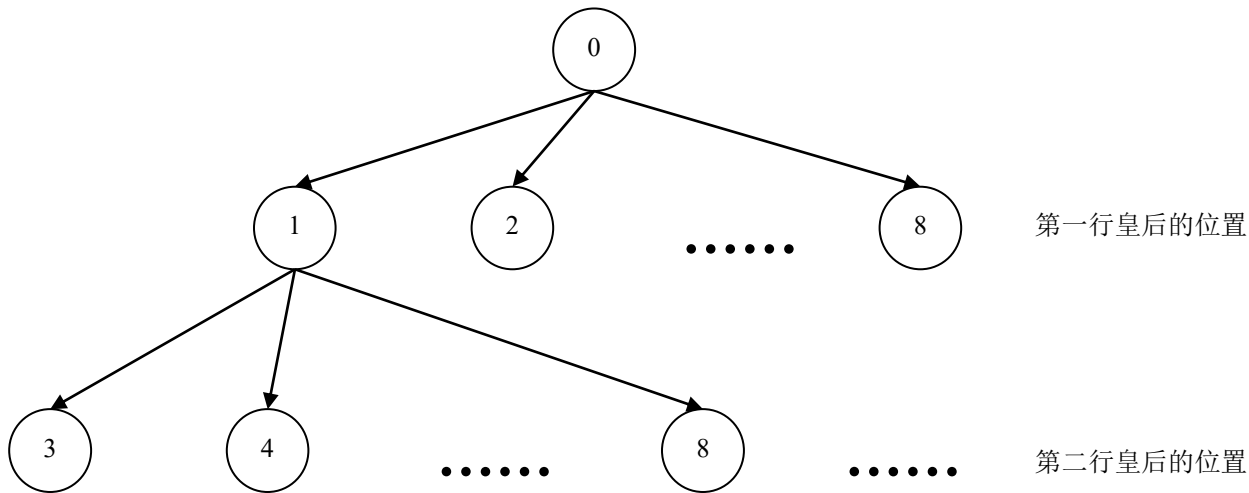
end;

思考：如果把递归加上，与刚才不加入递归的分析相比较。运行情况有什么不同？

每次确定皇后位置后，马上调用dfs(i+1)放置下一个皇后

注意：dfs(i+1)与dfs(i)是两个不同的程序，只不过名字和代码一样而已。

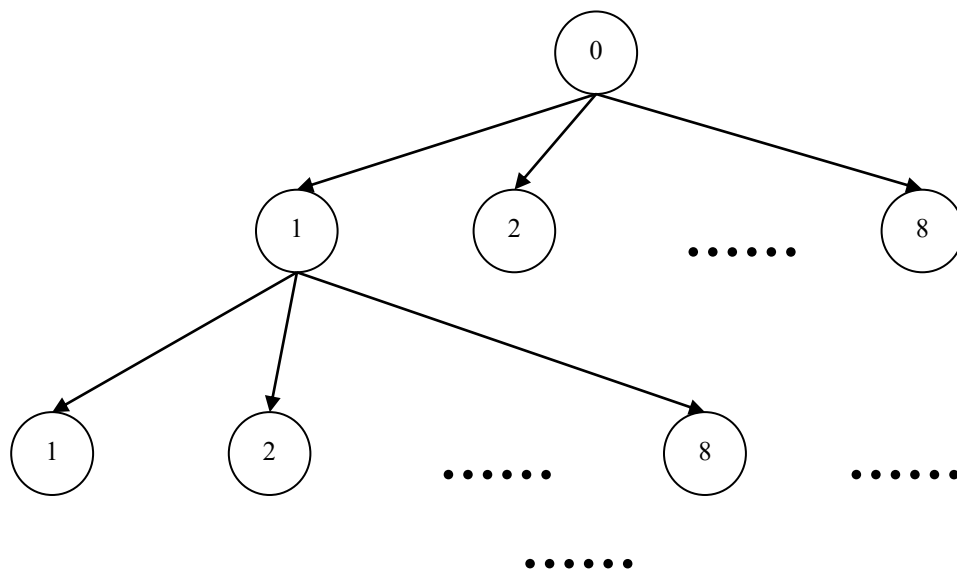
递归树



.....

- 所有的递归都是一棵递归树
- 递归遍历有三种顺序
 - 前序，中序，后序
- 多重循环是一棵树的前序遍历
- **DFS**算法是前序遍历
- 记忆化搜索是后序遍历
- **BST**排序输出是中序遍历

多重循环=前序遍历

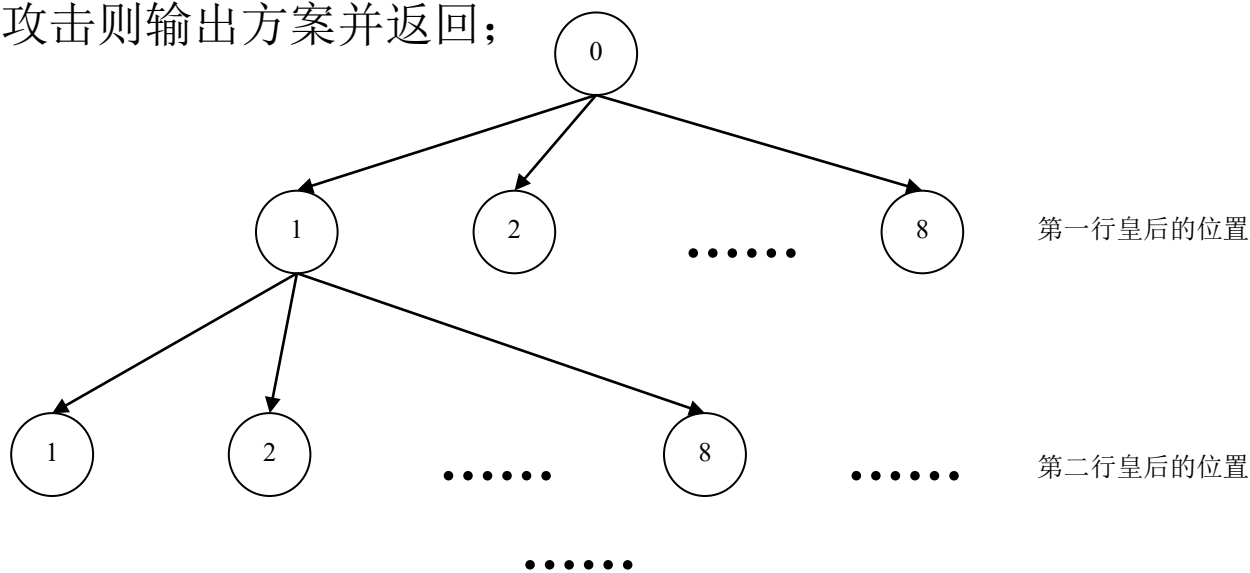


- For $a_1=1$ to 8 do
- for $a_2=1$ to 8 do
- for $a_3=1$ to 8 do
- ...
- if $a_1...a_8$ 互不攻击 then 输出该方案

不加剪枝的DFS =可控深度的多重循环

时间复杂度 $O(8^8)$
缺点：大量无效遍历

```
Procedure DFS(i:integer); //放第i个皇后
{搜索第i行皇后的位置}
Var
  j:integer;
begin
  if i=n+1 then 若互不攻击则输出方案并返回;
  for j:=1 to 8 do
  begin
    放置第i个皇后;
    DFS (i+1)
  end;
end;
```



DFS

时间复杂度远小于 $O(8!)$

原因：去掉了大量无效遍历（剪枝）

Procedure DFS(i:integer); //放第i个皇后
{搜索第i行皇后的位置}

Var

j:integer;

begin

if i=n+1 then 输出方案并返回;

for j:=1 to 8 do

begin

if 皇后能放在第i行第j列的位置 then

begin

放置第i个皇后;

对放置皇后的位置进行标记;

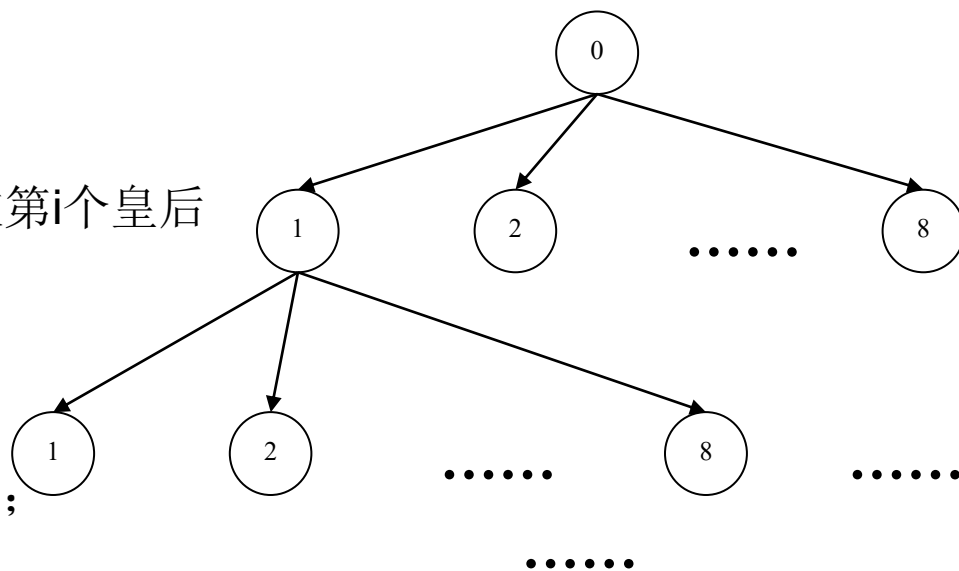
DFS (i+1)

对放置皇后的位置释放标记;

end;

end;

end;



第一行皇后的位置

第二行皇后的位置

DFS剪枝

- 可行性剪枝
- 最优性剪枝

```
Procedure DFS(i:integer); //放第i个皇后  
{搜索第i行皇后的位置}
```

```
Var
```

```
  j:integer;
```

```
begin
```

```
  if i=n+1 then 输出方案并返回;
```

```
  for j:=1 to 8 do
```

```
  begin
```

```
    if 皇后能放在第i行第j列的位置 then(可行性剪枝)
```

```
    begin
```

```
      放置第i个皇后; (放置方案保存在一个全局数组中)
```

```
      对放置皇后的位置进行标记;
```

```
      DFS (i+1)
```

```
      对放置皇后的位置释放标记;
```

```
    end;
```

```
  end;
```

```
end;
```

此处加最优性剪枝，若从现在开始已经比找到的最优解更差，则返回

练习：编码encode

只需从最低位到最高位枚举

枚举最i位时加上可行性剪枝即可

记忆化搜索

- `Dfs(i)`//确定节点*i*的最优值，必须是函数
- `Var`
- 枚举变量必须是局部容器
- `Begin`
- `if i是边界 then 返回`
- `if 最优值f(i)已计算过 then 返回`
- 枚举*i*的所有儿子找最优- $\rightarrow$ `f(i)`
- `返回f(i)`
-
- `End;`